

Designing a Model Predictive torque controller for a two wheeled balancing robot

Authors: Aryan Kulkarni, Diego Vangeneugden, Michael Rubin, Shanthanu Saravanan

ME C231A: Advanced Experiential Control 1
UC Berkeley
Fall 2025

Abstract—Two-wheeled inverted pendulum (TWIP) robots present a non-linear, unstable control problem. The system becomes significantly more complicated when inclined slopes or steering is implemented. This project applies Model Predictive Control (MPC) and evaluates system efficiency and stability under different operating conditions, specifically a perturbation balancing test, drive stop tests on different slopes and a circle steering test. To ensure real-time feasibility, the dynamic model is simplified by neglecting gyroscopic coupling. Performance is analyzed and optimized by tuning different MPC parameters. [Presentation Link](#)

I Introduction

Two-wheeled inverted pendulum robots have been a benchmark for controlling inherently unstable and non-linear systems. Stabilization on flat terrain has been extensively researched; however, introducing the objective of navigating inclined surfaces and executing directional changes adds additional layers of complexity to previously conducted research.

This project explores these challenges by integrating the slope and yaw steering directly into the control architecture. The primary objective is to understand the effect of the control parameters of MPC by analyzing system behaviour in different scenarios.

A dynamic model incorporating variable ground slope (ϕ) and yaw dynamics is derived, allowing the system to determine the correct pitch angle (θ_{eq}) to achieve equilibrium. The MPC problem is computed using CasADi and the system is simulated in MuJoCo and to validate the system performance. Future work includes bringing the controller to a physical robot, to gain further mechatronics experience.

The paper is organized as follows: Section II details the system dynamics; Section III formulates the MPC controller and Section IV analyzes different factors of the controller in various test scenarios.

II Dynamics

The dynamics for the two wheeled self-balancing robot consist of nonlinear coupled differential equations combining inverted pendulum dynamics for pitch stabilization, differential drive kinematics, and no-slip wheel constraints. A key contribution of our work is the inclusion of slope effects in conjunction with the full drive kinematic model.

State Space Model

The robot is modeled as a coupled system with state vector $x \in \mathbb{R}^6$ and control input $u \in \mathbb{R}^2$:

$$x = [x \quad \theta \quad \psi \quad \dot{x} \quad \dot{\theta} \quad \dot{\psi}]^T, \quad u = [\tau_L \quad \tau_R]^T$$

The state vector holds the following generalized coordinates: x is the linear position along the ground frame [m], θ is the pitch angle (body tilt) [rad], ψ is the heading angle [rad], $\dot{x}$ is the linear velocity [m/s], $\dot{\theta}$ is the pitch angular velocity or pitch rate [rad/s], $\dot{\psi}$ is the yaw rate [rad/s]. The control inputs τ_L, τ_R correspond to the left and right motor torques [Nm], which the robot's motor driver converts to voltage using the following relation:

$$V = \frac{\tau R}{k_t} + k_e \omega$$

where V is the applied voltage [V], τ is the desired torque [N·m], R is the winding resistance [Ω], k_t is the torque constant [N·m/A], k_e is the back-EMF constant [V·s/rad], and ω is the motor angular velocity [rad/s].

Equation of motion

The dynamic equation is solved in this form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \mathbf{F}_u - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{G}(\mathbf{q})$$

Mass Matrix M

$$\mathbf{M} = \begin{bmatrix} M_{11} & M_{12} & 0 \\ M_{12} & M_{22} & 0 \\ 0 & 0 & M_{33} \end{bmatrix}$$

$$M_{11} = m_{\text{eff}} = M_b + 2M_w + \frac{I_{wy}}{R^2}$$

$$M_{12} = M_b L \cos(\theta)$$

$$M_{22} = I_{\text{pitch}} = (M_b + M_w)L^2 + I_{by}$$

$$M_{33} = I_{\text{yaw}} = I_{wz} + I_{bx} \sin^2(\theta) - I_{bz} \cos^2(\theta)$$

In this model, the I_{yaw} term is significantly complicating the Mass matrix as it is varying with the pitch angle. For our purposes, considering steering (yaw) control is only implemented when the robot is relatively upright ($\theta \approx 0$), we can simplify this term by approximating it with a constant yaw inertia. Additionally, we will neglect the inertia of the wheels due to their negligible mass, relatively small size and

proximity to the yaw axis. This gives us the new simplified M33 term:

$$I_{yaw} = I_{bz}$$

much more suitable for real-time MPC because it provides faster solve times. The physical accuracy that is lost is well worth the computational load that is used.

Coriolis and Centrifugal Terms $\mathbf{C}$

$$\mathbf{C} = \begin{bmatrix} C_1 \\ C_2 \\ C_3 \end{bmatrix} = \begin{bmatrix} -M_b L \dot{\theta}^2 \sin(\theta) \\ -\frac{1}{2}(I_{bx} - I_{bz})\dot{\psi}^2 \sin(2\theta) \\ (I_{bx} - I_{bz})\dot{\theta}\dot{\psi} \sin(2\theta) \end{bmatrix}$$

Again, a significant simplification can be made to make our model more suitable for real-time MPC. As we won't be making any rapid turns (yaw rate remains small), we can assume planar motion dominates. Neglecting the gyroscopic effects means we can assume the pitch-yaw coupling terms (C_2 and C_3) are equal to zero.

Gravitational Terms $\mathbf{G}$

$$\mathbf{G} = \begin{bmatrix} (M_b + 2M_w)g \sin(\phi) \\ -M_b g L \sin(\theta + \phi) \\ 0 \end{bmatrix}$$

Generalized Forces $\mathbf{F}_u$

$$\begin{aligned} F_1 &= \frac{1}{R}(\tau_L + \tau_R) \quad (\text{longitudinal force}) \\ F_2 &= -(\tau_L + \tau_R) \quad (\text{pitch reaction torque}) \\ F_3 &= \frac{W}{2R}(\tau_R - \tau_L) \quad (\text{yaw torque}) \end{aligned}$$

CasADI solves EoM

We can restructure the equation of motion to the following form:

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{F}_u - \mathbf{C} - \mathbf{G})$$

which can be put in CasADI symbolic computation to construct the dynamics function for our controller.

Equilibrium on varying slopes

Equilibrium on slope with zero velocity

$$\begin{aligned} \mathbf{q}_{eq} &= [0, -\phi + \Delta\theta, 0, 0, 0, 0]^T \\ \boldsymbol{\tau}_{eq} &= \left[\frac{1}{2}M_{total} g \sin(\phi) r_{wheel}, \frac{1}{2}M_{total} g \sin(\phi) r_{wheel} \right]^T \end{aligned}$$

The equilibrium pitch $\theta_{eq} = -\phi + \Delta\theta$ compensates for ground slope as well as wheel torque and other unmodeled effects, achieving static balance slightly away from the geometric slope angle.

III Model Predictive Control

Cost Function

The MPC uses a quadratic cost function with separate state and control penalties:

Stage Cost

$$J = \sum_{k=0}^{N-1} [(x_k - x_{ref,k})^T Q (x_k - x_{ref,k}) + u_k^T R u_k]$$

Terminal Cost

$$J_N = (x_N - x_{ref,N})^T P (x_N - x_{ref,N})$$

Cost Matrices

- $Q \in \mathbb{R}^{6 \times 6}$ (**State Cost Matrix**): Diagonal matrix penalizing state deviations, the following starter values are implementing and will be tuned in the next chapter.
 - Highest weight on pitch angle θ (100.0) for balance priority
 - Moderate weights on pitch rate $\dot{\theta}$ (10.0) and yaw ψ (10.0)
 - Lower weights on position x (1.0) and velocities (1.0)
- $R \in \mathbb{R}^{2 \times 2}$ (**Control Cost Matrix**): Diagonal matrix penalizing control effort
 - Equal weights (0.1) on left and right wheel torques
 - Encourages smooth control actions
- $P \in \mathbb{R}^{6 \times 6}$ (**Terminal Cost Matrix**): Computed via Discrete Algebraic Riccati Equation (DARE)
 - $P = A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d + Q$
 - Provides closed-loop stability guarantees
 - Uses discretized system matrices A_d, B_d at sampling period T_s

Constraints

There is a state constraint on the pitch ($|\theta| \leq 30$) and pitch rate ($|\dot{\theta}| \leq 3.51 \text{ rad/s}$), to ensure the robot remains within the operational limits and never leans on the safety rails. A safety mechanism is also put in place: when pitch reaches 30° or above, meaning the robot has definitely fallen over beyond recovery, all control actions are stopped.

Furthermore, there are symmetric torque limits for both motors:

$$|\tau_i| \leq \tau_{\max} \quad \text{for } i \in \{1, 2\}$$

$\tau_{\max}$ for the tests in this project was set at 0.25Nm. This ensures actuator saturation is respected and the hardware limitations are respected. The pitch rate constraint also ensures feasibility with respect to this saturation.

Horizon Length

The prediction horizon N must be long enough to capture pitch stabilization dynamics while keeping computational cost achievable by using a realistic sampling period T_s . The total duration of the horizon is defined as $N \times T_s$ seconds.

Complete Receding Horizon Problem

The MPC solves the following optimization problem at each time step:

$$\begin{aligned}
 \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} [(x_k - x_{ref,k})^T Q (x_k - x_{ref,k}) + u_k^T R u_k] + (x_N - x_{ref,N})^T P (x_N - x_{ref,N}) \quad (1a) \\
 \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \quad k = 0, \dots, N-1 \quad (1b) \\
 & x_0 = x(t) \quad (1c) \\
 & |\theta_k| \leq \theta_{\max}, \quad k = 1, \dots, N \quad (1d) \\
 & |\dot{\theta}_k| \leq \dot{\theta}_{\max}, \quad k = 1, \dots, N \quad (1e) \\
 & |u_{k,i}| \leq \tau_{\max}, \quad k = 0, \dots, N-1, i \in \{1, 2\} \quad (1f)
 \end{aligned}$$

where $A \in \mathbb{R}^{6 \times 6}$ and $B \in \mathbb{R}^{6 \times 2}$ are the discrete-time linearized system matrices.

This MPC formulation balances the competing objectives of maintaining vertical balance (pitch regulation), tracking desired trajectories (x, y position), and controlling heading (yaw) while respecting physical actuator and geometrical limits. Results were simulated in Mujoco as well as tested on the physical robot.

IV MPC Design Parameters

Objectives and performance metrics

Primary Objectives:

- 1) Balance Stabilization: Maintain $|\theta| < 5^\circ$ during operation
- 2) Position Tracking: Follow desired (x, y) trajectory with error < 10 cm
- 3) Heading Control: Track desired heading ψ with error $< 5^\circ$

Secondary Objectives:

- 4) Smooth control inputs (minimize Δu)
- 5) Energy efficiency
- 6) Disturbance rejection (pushes, ground irregularities)

Sampling period and prediction horizon size

A smaller sampling period T_s will enhance accuracy and ensure disturbances are handled fast enough, but will require more compute. An open loop test was designed to define the 10% to 90% rise time to reach the final steady-state value (1.3s). This helped identify an appropriate sampling time, as literature recommends placing 20 samples within this rise time [1]. This analysis yields a sampling time of 0.065s, which translates to a control frequency of around 15Hz. We realize this may be on the slow side for actual implementation, but simulations show promising results (Fig.1).

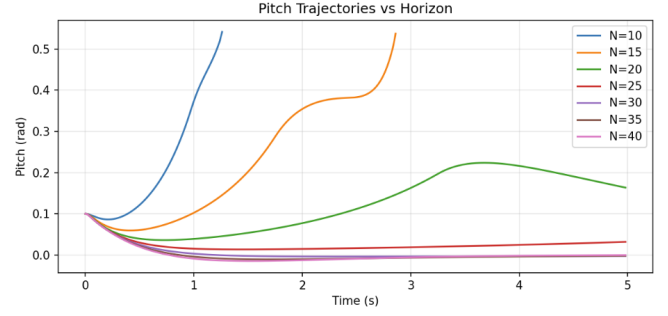
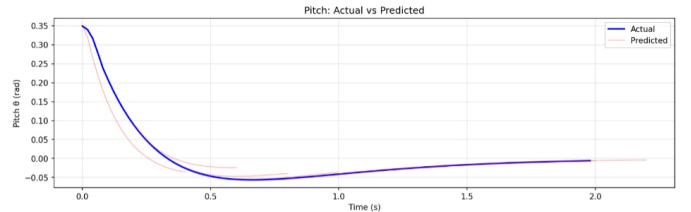


Fig. 1: Pitch horizon trajectories

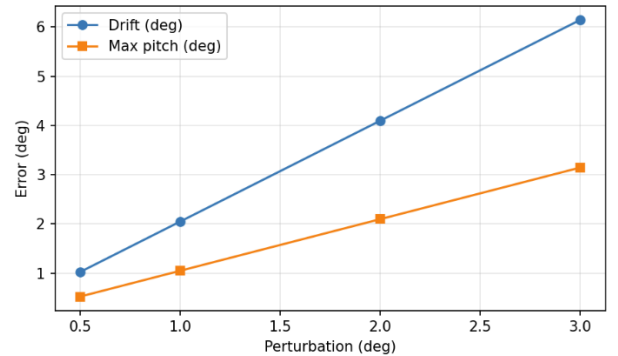
The prediction horizon N defines how far into the future our controller solves the optimal control problem at each iteration. An insufficient horizon length leads to a short-sighted controller that fails to anticipate future state trajectories, thus resulting in infeasibility. A longer horizon enables more anticipatory behavior and smoother trajectories, but increases computational cost since only the first control input is implemented at each iteration. From a stability perspective, the horizon must be long enough to capture the dominant system dynamics and ensure feasibility of terminal constraints; insufficient horizon length can compromise closed-loop stability. A prediction horizon of $N = 30$ samples was selected to maintain real-time feasibility while maximizing control performance with favorable computational trade-offs. [2]

Cost function matrices and MPC performance

To investigate the influence of cost function weighting, multiple test scenarios were conducted with varying Q and R matrix tunings to assess controller performance.



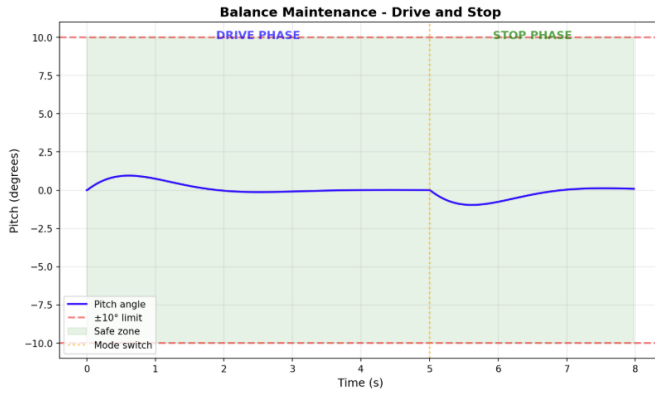
(a) MPC prediction accuracy



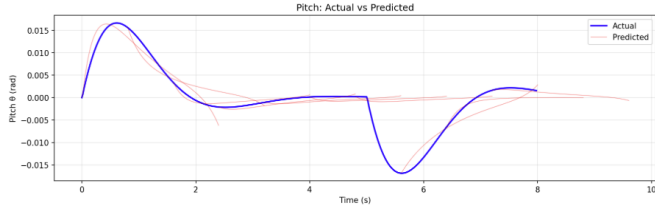
(b) perturbation recovery

Fig. 2: Standard balancing test

Simple balancing tests as well as perturbation recovery tests were successful, with the robot reaching acceptable maximum angle deviations (drift) as seen in Figure 2. As expected, pitch and pitch rate dominate matrix weights in the best performing tests, prioritizing upright stabilization against disturbances over other states. Position, velocity, and yaw terms receive minimal emphasis since stationary equilibrium decouples lateral and translational dynamics. Where the pitch angle changes gradually and remains relatively flat, the MPC predictions closely follow the actual trajectory, indicating accurate model representation. However, the controller struggles with rapid transients, as can be seen by the prediction accuracy in Fig. 2a which is worse at the start when the angle from equilibrium is largest. This discrepancy suggests that improving the linearization of the system dynamics, particularly around larger deviations from equilibrium, would enhance prediction accuracy during aggressive maneuvers. This is further analyzed in subsection *“Different linearization methods”*.



(a) Pitch correction



(b) MPC prediction accuracy

Fig. 3: Drive-stop test

In the drive-stop the controller shows strong steady-state tracking with clear opportunities for improvement during transient events. This is consistent with the results from the balancing and disturbance testing. It has to be noted the initial divergence at the state transition from driving to balancing mode (around 4-5 seconds) is expected behavior, as the MPC cannot anticipate this discrete state change and must reactively adjust its internal model.

In the drive-stop tests position and velocity receive the heaviest weights to enable precise forward tracking and smooth deceleration. Pitch and pitch rate get moderate emphasis for lean compensation during acceleration, while yaw stays lightly weighted as there is no turning or steering involved.

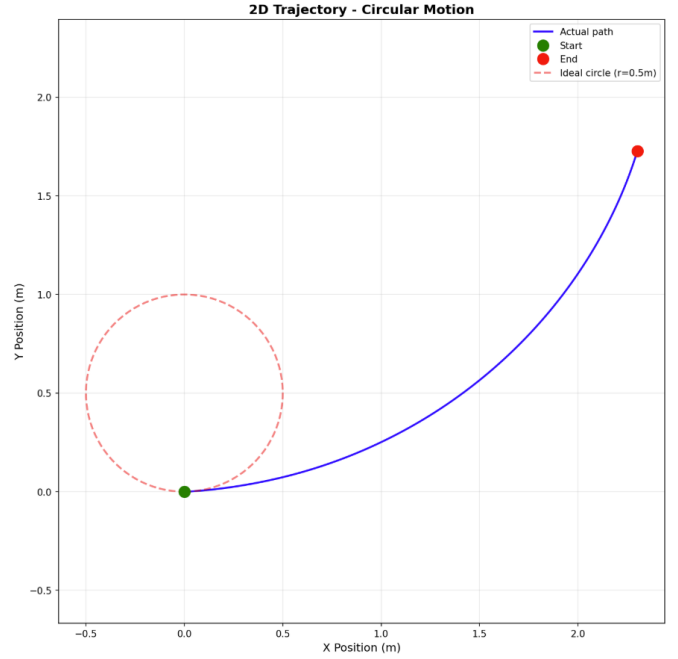


Fig. 4: Circle driving trajectory

For circle driving tests, yaw and yaw rate dominate the weights to maintain consistent turning radius against centrifugal instabilities. Position/velocity balance path adherence with moderate priority, while pitch terms stay secondary as steady circular motion minimizes sagittal perturbations. Heavy prioritization (up to 500%) was implemented but steady yaw could not be achieved, which is portrayed in Figure 4. However, open-loop yaw test were successful, ruling out fundamental issues with simulation physics, actuator models, or basic differential drive kinematics. The MPC failing to drive in circles despite valid yaw torque commands highly likely stems from model-simulation mismatches in the linearized state-space dynamics such as unmodeled wheel slip angles, ground friction coupling or neglected pitch-yaw cross terms. These issues remain unresolved.

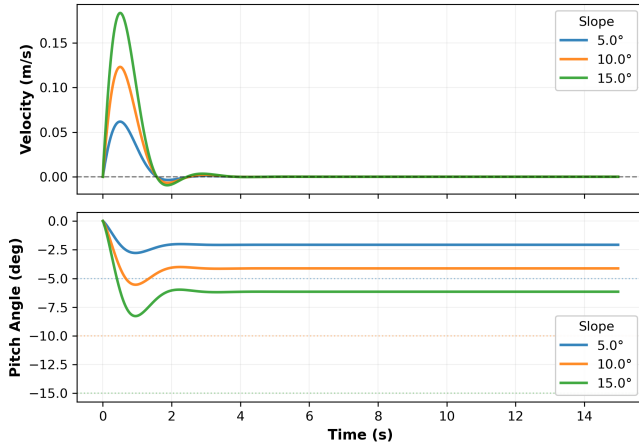


Fig. 5: Stabilization on different slopes

Slope stabilization tests were successful and the results match our expectations. For slope stabilization position and pitch rate dominate matrix weights, prioritizing stabilization against gravity-induced torques over other states. Pitch and velocity receive moderate emphasis to enable controlled uphill lean without drift, while yaw terms stay lightly weighted due to induced yaw moments. Figure 5 shows that the robot settles at a pitch that is less angled than the slope angle, while the velocity converges to zero. This behavior is expected: to hold position on an incline, the wheels must generate an uphill torque that balances the downhill component of gravity, which requires the body to lean slightly further uphill than the slope itself, leading to a steady-state angle offset as seen in the plots.

Effectiveness of warm start

Warm starting is an optimization technique that accelerates solver convergence by initializing each MPC iteration with information from the previous solution rather than starting from scratch. When enabled, our solver uses the shifted optimal control sequence from the previous time step as an initial guess. This allows for a faster solve time, improving the real-time compatibility of the controller. Figure 6 compares the solve times of cold vs warm starting.

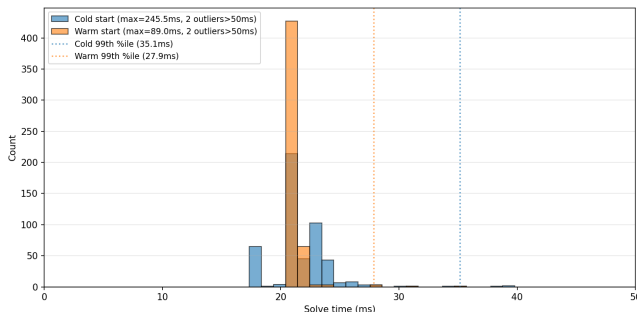


Fig. 6: Solve time histogram: cold vs warm start

Since this controller will be applied to a physical, real-time system, not just the average solving time is important but the worst case also is of concern. Our worst-case solve time improved by 63.8% by utilizing a warm start approach, the 99th percentile solve time improved by 20.5%.

Different linearization methods

Originally the model was objectively linearized around pitch zero. The more the pitch deviated the bigger linearization error. By linearizing the model around the current state (online relative linearization), we expected the controller to achieve stabilization up to a higher maximal speed. However, the simulations showed poor results, failing faster than with objective linearization (at 15° pitch).

Our hypothesis is that this issue arises due to the simplified dynamic model that is in place. As the robot tilts, the model assumptions break down, so linearizing online does not accurately capture the true behavior away from equilibrium. It only results in more compute as the jacobian has to be constantly re-evaluated. Consequently, using the full nonlinear dynamics in the controller was not feasible due to inefficient solve times.

V Conclusion and future work

This work successfully implements an MPC-based controller for a self-balancing robot with slope compensation, while identifying key areas for future enhancement. A wide range of MPC parameters was investigated, deepening our understanding of MPC and enabling better controller tuning. For successful yaw and steering control, improving model accuracy while considering computational efficiency should be explored. To ensure feasibility while enlarging the operating region, terminal constraints combined with a control invariant sets (CIS) for the terminal region could yield better high-velocity performance. For physical implementation, optimizing MPC without accurate state estimation is not meaningful. Integrating a Kalman filter (or similar observer) to fuse sensor data with the model would reduce noise and provide reliable state estimates for MPC when used in a real system. Extensive tuning of MPC weights and constraints is of limited value without equal attention to state estimation.

Other controllers, such as PID, were not evaluated since that was beyond the scope of the project for this course. When designing a controller for a system, a consideration for which controller to use should always be made. Advantages of MPC in this project were the implicit state and input constraints we could implement, a drawback was the higher compute which cannot be run on a low-level microcontroller such as an arduino.

Acknowledgements

We would like to thank THE GOAT Dr. Francesco Borelli for introducing this fascinating subject in the ME C231A lectures in such an inspiring fashion. The support of Akshay Thirugnanam and Fiona Kopp throughout the semester have also allowed us to dive as deep into the subject as we have.

It should be noted that the use of an AI agent was made to speed up the software development and ensure modularity of our code. This allowed us to design more tests and gain more insight on the matter, as design parameters were more easily changed. We acknowledge the code structure would have been less advanced without the use of these AI tools.

Statement of work

All authors contributed equally to the work required throughout the project. The authors are listed alphabetically.

References

- [1] MathWorks, “Understanding model predictive control, part 3: Mpc design parameters,” Video, 2018, accessed: 2025-12-12. [Online]. Available: <https://www.mathworks.com/videos/understanding-model-predictive-control-part-3-mpc-design-parameters-1530607670393.html>
- [2] —, “Choose sample time and horizons - MATLAB & Simulink,” 2025, accessed: 2025-12-12. [Online]. Available: <https://www.mathworks.com/help/mpc/ug/choosing-sample-time-and-horizons.html>
- [3] N. Keller, J. Schilliger, and Y. Schnider, “Predictive control of a two-wheeled balancing robot: Lab practice control systems - experiment script,” Institute for Dynamic Systems and Control, ETH Zurich, Zurich, Switzerland, Laboratory Experiment Script, Feb. 2024. [Online]. Available: <https://ethz.ch/content/dam/ethz/special-interest/mavt/dynamic-systems-n-control/idsc-dam/Lectures/Control-Lab/SigiStudent.pdf>
- [4] P. J. Goulart, E. C. Kerrigan, and J. M. Maciejowski, “Efficient model predictive control with warm starting and custom quadratic program solvers,” in *Proceedings of the UK Automatic Control Conference (UKACC)*. Glasgow, UK: Imperial College London, 2006.
- [5] Y. Wang and S. Boyd, “Fast model predictive control using online optimization,” *IEEE Transactions on Control Systems Technology*, vol. 18, no. 2, pp. 267–278, Mar. 2010, discusses acceleration of online optimization via warm starting, interior-point methods, and early termination.
- [6] T. Marcucci and R. Tedrake, “Warm start of mixed-integer programs for model predictive control of hybrid systems,” in *2019 IEEE 58th Conference on Decision and Control (CDC)*. Nice, France: IEEE, 2019, pp. 3770–3777. [Online]. Available: https://groups.csail.mit.edu/robotics-center/public_papers/Marcucci19.pdf
- [7] F. Borrelli, “Lecture 4: Optimal Control,” Lecture slides, ME C231A: Advanced Experiential Control 1, University of California, Berkeley, Oct. 2025, accessed: 2025-12-19.
- [8] —, “Lecture 5: Model Predictive Control,” Lecture slides, ME C231A: Advanced Experiential Control 1, University of California, Berkeley, Dec. 2025, accessed: 2025-12-19.

VI Appendix

Appendix A: Hardware specifications

Primary Hardware

Robot Platform: Pololu Balboa 32U4 Balancing Robot Kit (Product #3575)

- Onboard ATmega32U4 microcontroller
- Two 50:1 HP micro metal gearmotors with encoders
- LSM6DS33 6-axis IMU (accelerometer + gyroscope)
- 80×10 mm wheels
- TB6612FNG dual motor driver

Computation Unit: Raspberry Pi 4 Model B (4GB RAM)

- Quad-core ARM Cortex-A72 @ 1.5 GHz
- Communication via USB or UART with Balboa 32U4

Sensor Suite

Inertial Measurement: LSM6DS33 IMU

- 3-axis accelerometer ($\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$)
- 3-axis gyroscope ($\pm 125^\circ/s$ to $\pm 2000^\circ/s$)
- I²C interface to 32U4

Wheel Encoders: Magnetic encoders on both motors

- 12 counts per revolution of motor shaft
- 600 counts per revolution of wheel (with 75:1 gearing)
- Quadrature encoding for direction sensing

Actuation

Motors: Two independently controlled DC motors

- Voltage: 6V nominal (max 6V)
- Speed: ~ 150 RPM at 6V (no load)
- Torque: ~ 35 oz-in (0.25 N·m) at 6V stall
- PWM control via TB6612FNG driver

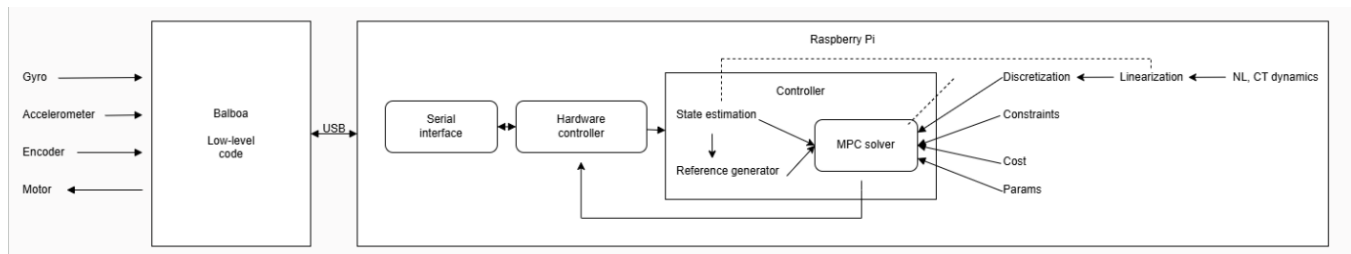


Fig. 7: hardware and code architecture

Appendix B: Firmware development

High-Level Controller (Raspberry Pi)

- Python 3.9+S
- MPC solver: CasADi
- State estimator: EKF or complementary filter

Low-Level Controller (ATmega32U4)

- Arduino-compatible C/C++
- Motor PWM generation
- Encoder interrupt handling
- IMU data filtering